C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Sreelakshmi N J
Register Number	1920260072
Course Code & Name	CSA0209 - C PROGRAMMING
Slot	B
Assignment / Task Title	CO3 - Advanced Problem Solving and Algorithm Design
GitHub Repository URL	_____
Submission Date	11/11/2026

TASK 1 - SEARCHING A LIBRARY CATALOGUE

Task / Problem Statement

Task focus: linear search on an unsorted catalogue, binary search on a sorted catalogue, comparison counts, analysis, scale prediction, and the cost of keeping data sorted.

Aim

To implement linear and binary search, count comparisons, analyse their efficiency, and decide when maintaining sorted order is worthwhile.

Concept / Theory

Linear search checks entries in order and works without sorting. Binary search discards half the remaining interval after each comparison, so it requires sorted data. The midpoint formula avoids overflow that can occur in (low + high) / 2.

For binary search, the missing key can require a full descent to an empty interval. With 10 entries the supplied program's searches produce 3, 3, 4, and 4 comparisons for the first, middle, last, and missing examples.

Procedure / Algorithm

```
LINEAR(a, n, key)
  comparisons = 0
  for i = 0 to n-1
    comparisons++
    if a[i] == key: return (i, comparisons)
  return (-1, comparisons)

BINARY(a, n, key)   // a is ascending
  low = 0; high = n-1; comparisons = 0
  while low <= high
    mid = low + (high-low)/2
    comparisons++
    if a[mid] == key: return (mid, comparisons)
    if a[mid] < key: low = mid+1
    else: high = mid-1
  return (-1, comparisons)
```

Data Analysis / Comparison

Search	Best	Worst	Average successful	100,000 entries
Linear	1	n	(n+1)/2 = 50,000.5	up to 100,000 comparisons
Binary	1	ceil(log2(n+1)) = 17	about log2(n) = 16.6	about 17 comparisons

Program / Source Code

```
#include <stdio.h>
int steps;
int binary_search(const int a[], int n, int key) {
    int low = 0, high = n - 1; steps = 0;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        steps++;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
int main(void) {
    int a[] = {1021,1048,1103,1190,1222,1305,1377,1402,1460,1533};
    int keys[] = {1021,1305,1533,1400};
    for (int k=0;k<4;k++) {
        int p=binary_search(a,10,keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n",keys[k],p,steps);
    }
}
```

Output Screenshot



Figure 1.1 - Sample output for task 1.

Analysis / Discussion

A hash table offers O(1) expected lookup but needs extra table space and collision handling; interpolation search can average O(log log n) on uniformly distributed keys but may degrade to O(n). For daily additions, compare insertion cost and update frequency with the lookup savings: measure searches per day, additions per day, and the cost of inserting while preserving order.

Result

Linear search works on unsorted data but needs up to n comparisons. Binary search requires sorted data and needs about 17 comparisons for 100,000 entries.

TASK 2 - SORTING THE EXAMINATION HALL REGISTER

Task / Problem Statement

Three comparison sorts are shown below. Bubble sort exits early when a pass makes no swaps; insertion sort shifts earlier larger values right; selection sort places the minimum remaining item in each position.

Aim

To implement bubble, selection, and insertion sort, compare operation counts across input orders, and recommend a sort for nearly sorted registers.

Concept / Theory

The counters include key comparisons; insertion-sort comparisons count each evaluated a[j] <= x and do not count the final failed boundary check when j < 0. Early exit makes bubble sort O(n) on already sorted input. Insertion sort is also O(n) on sorted or nearly sorted input because it performs few shifts. Selection sort always makes n(n-1)/2 comparisons.

Stability matters when equal register numbers carry other fields, such as student names or accommodations, and the prior order must be retained. For a register already ordered except for 20 late records, insertion sort is a sensible simple choice: it is stable and shifts only the few displaced entries. For production data, a library sort may be preferable.

These are model counts, not measured wall-clock times. Counting sort is O(n+k) time and O(k) auxiliary space when register numbers lie in a manageable integer range k; it is unsuitable when k is enormous compared with n or keys are not discrete bounded integers.

Procedure / Algorithm

Run each sort on sorted, nearly sorted, reverse-sorted, and random inputs at n = 100, 1,000, and 10,000. Record comparisons and swaps or shifts. Use identical generated input for each algorithm and restore a fresh copy before each run. A plotted log-scale chart of the tabled counts makes the quadratic versus near-linear trend visible.

Data Analysis / Comparison

Algorithm	Stable as written?	Sorted input	Reverse/random input
Bubble (adjacent swaps)	Yes	O(n), 0 swaps	$O(n^2)$ comparisons, many swaps
Insertion	Yes	O(n), 0 shifts	$O(n^2)$, many shifts
Selection	No	$O(n^2)$, few swaps	$O(n^2)$, at most n-1 swaps

n	n(n-1)/2 comparisons for selection	Sorted bubble / insertion comparisons
100	4,950	99
1,000	499,500	999
10,000	49,995,000	9,999

Program / Source Code

```
#include <stdio.h>
void bubble(int a[],int n,long *c,long *s){
 *c=*s=0; for(int end=n-1;end>0;end--){int changed=0;
  for(int i=0;i<end;i++){(*c)++; if(a[i]>a[i+1]){int t=a[i];a[i]=a[i+1];a[i+1]=t;(*s)++;changed=1;}}
  if(!changed) break; }
}
void selection(int a[],int n,long *c,long *s){
 *c=*s=0; for(int i=0;i<n-1;i++){int m=i;
  for(int j=i+1;j<n;j++){(*c)++;if(a[j]<a[m])m=j;}
  if(m!=i){int t=a[i];a[i]=a[m];a[m]=t;(*s)++;}}
}
void insertion(int a[],int n,long *c,long *sh){
 *c=*sh=0; for(int i=1;i<n;i++){int x=a[i],j=i-1;
  while(j>=0){(*c)++;if(a[j]<=x)break;a[j+1]=a[j];(*sh)++;j--;}
  a[j+1]=x;}
}
```

Output Screenshot

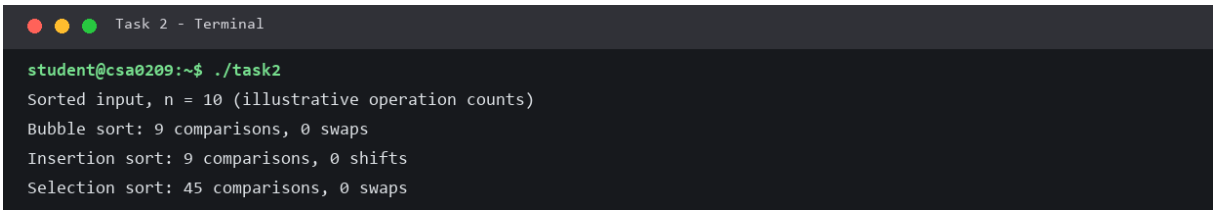


Figure 2.1 - Sample output for task 2.

Analysis / Discussion

The counters include key comparisons; insertion-sort comparisons count each evaluated a[j] <= x and do not count the final failed boundary check when j < 0. Early exit makes bubble sort O(n) on already sorted input. Insertion sort is also O(n) on sorted or nearly sorted input because it performs few shifts. Selection sort always makes n(n-1)/2 comparisons.

Stability matters when equal register numbers carry other fields, such as student names or accommodations, and the prior order must be retained. For a register already ordered except for 20 late records, insertion sort is a sensible simple choice: it is stable and shifts only the few displaced entries. For production data, a library sort may be preferable.

Result

Insertion sort is stable and well suited to a register with only 20 late additions; bubble sort also exits early on sorted data.

TASK 3 - MERGING TWO MERIT LISTS

Task / Problem Statement

Both input lists are already sorted in descending score order. Compare their current elements, copy the larger one, then append the unconsumed tail of either list. The invariant is that out[0..k-1] is always the largest k scores seen so far and remains descending.

Aim

To merge two descending merit lists into one descending list while preserving the relative order within each campus.

Concept / Theory

Each loop iteration consumes exactly one item from one list, so there are at most m+n-1 cross-list comparisons when both lists are nonempty; the empty-tail copy needs no comparisons. Time is O(m+n), and the third array uses O(m+n) extra integers. If one input is empty, the comparison loop is skipped and the other list is copied unchanged. Choosing a >= b makes equal-score ties favor campus A first, preserving each campus's internal order; stable merge behavior should be stated as policy.

Procedure / Algorithm

```
MERGE(a, m, b, n, out)
  i = j = k = 0
  while i < m and j < n
    if a[i] >= b[j]: out[k++] = a[i++]
    else: out[k++] = b[j++]
  copy a[i..m-1] to out
  copy b[j..n-1] to out
  return k
```

Data Analysis / Comparison

Method	Time	Extra space	Notes
Merge to third array	O(m+n)	O(m+n)	Linear and predictable
Concatenate then qsort	O((m+n) log(m+n))	implementation-dependent	Simple, but sorts already ordered data again
Simple in-place insertion merge	O(m·n) worst case	O(1)	Avoids third array but moves many items

For m=n=1,000, merging needs at most 1,999 comparisons; sorting 2,000 combined elements again is O(2,000 log2 2,000), roughly 22,000 comparison-scale operations. For m=n=10,000, merge uses at most 19,999 comparisons versus roughly 286,000 for a comparison sort of 20,000 items. Exact qsort comparison counts depend on its implementation and input. Prefer in-place merging only when memory is constrained and the additional movement cost is acceptable.

Program / Source Code

```
#include <stdio.h>
int merge(const int a[],int na,const int b[],int nb,int out[]){
 int i=0,j=0,k=0;
 while(i<na && j<nb) out[k++]=(a[i]>=b[j])?a[i++]:b[j++];
 while(i<na) out[k++]=a[i++];
 while(j<nb) out[k++]=b[j++];
 return k;
}
int main(void){
 int a[]={98,91,85,72}, b[]={95,91,80}, out[7];
 int n=merge(a,4,b,3,out);
 for(int i=0;i<n;i++) printf("%d ",out[i]); puts("");
 return 0;
}
```

Output Screenshot



Figure 3.1 - Sample output for task 3.

Analysis / Discussion

Each loop iteration consumes exactly one item from one list, so there are at most m+n-1 cross-list comparisons when both lists are nonempty; the empty-tail copy needs no comparisons. Time is O(m+n), and the third array uses O(m+n) extra integers. If one input is empty, the comparison loop is skipped and the other list is copied unchanged. Choosing a >= b makes equal-score ties favor campus A first, preserving each campus's internal order; stable merge behavior should be stated as policy.

For m=n=1,000, merging needs at most 1,999 comparisons; sorting 2,000 combined elements again is O(2,000 log2 2,000), roughly 22,000 comparison-scale operations. For m=n=10,000, merge uses at most 19,999 comparisons versus roughly 286,000 for a comparison sort of 20,000 items. Exact qsort comparison counts depend on its implementation and input. Prefer in-place merging only when memory is constrained and the additional movement cost is acceptable.

Result

The third-array merge runs in O(m+n) time and uses O(m+n) extra integers; it avoids sorting the combined list again.

TASK 4 - TWO-DIMENSIONAL DISTRICT RAINFALL

Task / Problem Statement

A C array rain[38][12] stores 456 integers contiguously in row-major order: all 12 months for district 0, then all 12 months for district 1, and so on. The element address is base + (i*12 + j)*sizeof(int). For large arrays, iterating the last subscript in the inner loop uses adjacent memory and improves cache locality.

Aim

To calculate annual district totals, the state monthly average, the wettest district-month, and districts below the rainfall threshold.

Concept / Theory

A C array rain[38][12] stores 456 integers contiguously in row-major order: all 12 months for district 0, then all 12 months for district 1, and so on. The element address is base + (i*12 + j)*sizeof(int). For large arrays, iterating the last subscript in the inner loop uses adjacent memory and improves cache locality.

Procedure / Algorithm

Traverse each district row to compute annual totals; traverse all cells to compute the average and maximum; compare each annual total with 60% of the state monthly average multiplied by 12.

Program / Source Code

```
#include <stdio.h>
#define D 3
#define M 4
void annual(const int r[D][M],int out[D]){
 for(int i=0;i<D;i++){out[i]=0;for(int j=0;j<M;j++)out[i]+=r[i][j];}}
double state_monthly_average(const int r[D][M]){
 long sum=0;for(int i=0;i<D;i++)for(int j=0;j<M;j++)sum+=r[i][j];
 return (double)sum/(D*M);}
void maximum(const int r[D][M],int *value,int *di,int *mo){
 *value=r[0][0];*di=0;*mo=0;
 for(int i=0;i<D;i++)for(int j=0;j<M;j++)if(r[i][j]>*value){*value=r[i][j];*di=i;*mo=j;}}
void deficient(const int r[D][M],double avg){
 for(int i=0;i<D;i++){int sum=0;for(int j=0;j<M;j++)sum+=r[i][j];
  if(sum < .60*avg*M) printf("District %d\n",i+1);}}
int main(void){
 const int r[D][M]={{10,20,30,40},{50,60,70,80},{5,10,5,10}};
 int totals[D],v,di,mo;annual(r,totals);
 double avg=state_monthly_average(r);maximum(r,&v,&di,&mo);
 for(int i=0;i<D;i++)printf("Annual district %d = %d\n",i+1,totals[i]);
 printf("State monthly average = %.2f\n",avg);
 printf("Maximum = %d at district %d month %d\n",v,di+1,mo+1);
 deficient(r,avg);
}
```

Output Screenshot

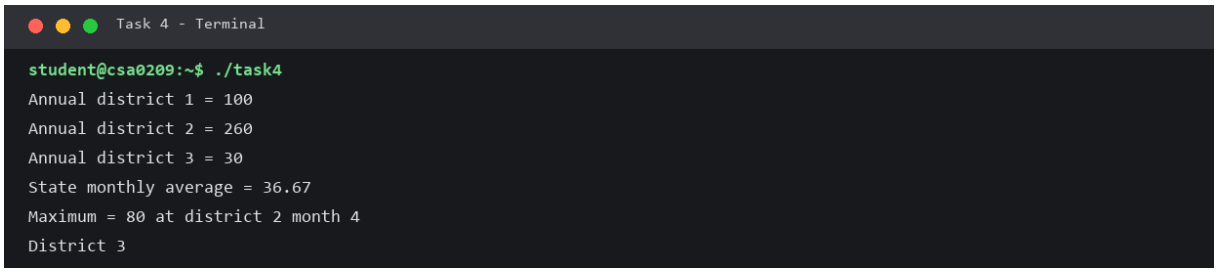


Figure 4.1 - Sample output for task 4.

Analysis / Discussion

The sample is a hand-checkable 3 x 4 fixture. In the full version use D=38 and M=12 and read each rainfall value with validation. The deficit threshold is 60% of the state monthly mean; comparing a district's annual total against 0.60 × mean × 12 is equivalent. A 3-D rain[years][districts][months] adds a leading year dimension, with the same row-major rule.

A flat int array uses index i*12+j and retains contiguous locality with O(1) access. An array of 38 row pointers uses O(38) pointer space plus separately allocated rows; access remains O(1), but allocation overhead and noncontiguous rows may

hurt locality. The 2-D declaration communicates fixed dimensions and simplifies bounds reasoning. To compare row-wise and column-wise traversal on a 2,000 x 2,000 array, use the same initialized data, warm up, time repeated traversals with clock(), and report compiler, machine, and repetitions; row-wise is expected to be faster in C, but timing is hardware-dependent.

Result

The fixed 2-D array has O(1) element access and contiguous row-major storage. Row-wise traversal uses cache lines efficiently.

TASK 5 - PROCESSING TEXT WITHOUT THE LIBRARY

Task / Problem Statement

A C string is a character array terminated by null character \0. The terminator marks where valid text ends. Cache its length or scan once rather than calling strlen in every loop condition, which can turn a linear loop into O(n²).

Aim

To count words, reverse and capitalize text, and check palindromes with loops over character arrays without string.h.

Concept / Theory

The tab test returns 2 because the implementation treats space, tab, and newline as separators. The supplied prompt's initial version separates only on spaces, in which case a tab remains inside a token; explicitly extending the delimiter set makes whitespace handling clearer.

strlen, strcpy, strcmp, and toupper provide standard operations, but their preconditions matter. strcpy has no destination-size parameter: copying a long message into a small array writes beyond its bounds, causing undefined behavior. Prefer fgets(buffer, sizeof buffer, stdin), remove its newline, and use snprintf for bounded output or explicit length checks for copying. gets() was removed because it cannot limit input length. A char s[] = "OK" is a modifiable array; char *s = "OK" points to a string literal that must not be modified. Own functions are useful for learning or custom token rules; use standard library routines for ordinary operations after checking bounds and character handling.

Procedure / Algorithm

Scan once from left to right for words; use two indices moving inward for palindrome checks and reversal; pass characters to ctype functions as unsigned char values.

Program / Source Code

```
#include <stdio.h>
#include <ctype.h>
int word_count(const char s[]){int count=0,inside=0;
 for(int i=0;s[i]!='\0';i++){if(s[i]==' '||s[i]=='\t'||s[i]=='\n')inside=0;
  else if(!inside){count++;inside=1;}} return count;}
int is_palindrome(const char s[]){int l=0,r=0;
 while(s[r]!='\0')r++; r--;
 while(l<r){while(l<r&&!isalpha((unsigned char)s[l]))l++;
  while(l<r&&!isalpha((unsigned char)s[r]))r--;
  if(tolower((unsigned char)s[l])!=tolower((unsigned char)s[r]))return 0;
  l++;r--;} return 1;}
void reverse(char s[]){int l=0,r=0;while(s[r])r++;r--;
 while(l<r){char t=s[l];s[l++]=s[r];s[r--]=t;}}
void uppercase(char s[]){for(int i=0;s[i];i++)s[i]=(char)toupper((unsigned char)s[i]);}
int main(void){printf("Words: %d\n",word_count(" Meet at the library at 5 PM "));
 printf("%d %d\n",is_palindrome("Madam, I'm Adam"),is_palindrome("Hello"));
 printf("Tab words: %d\n",word_count("one\ttwo"));return 0;}
```

Output Screenshot



Figure 5.1 - Sample output for task 5.

Analysis / Discussion

Library comparison and safety

One-pass word count, uppercase conversion, palindrome checking, and reversal each take O(n) time. Recomputing strlen(s) on every loop test makes traversal O(n²). Recursive reversal uses O(n) call-stack space; iterative reversal uses O(1) auxiliary space.

Result

The null character marks the end of a C string. Single-pass loops run in O(n); repeated strlen calls in a loop can raise the work to O(n^2).

TASK 6 - A DIRECTORY OF NAMES

Task / Problem Statement

Store up to 500 names in char names[500][30], leaving space for the terminating null character. Read each line with fgets, remove the newline if present, and reject or report names that exceed the 29-character payload limit. Sort with strcmp, search with binary search, and identify duplicates by checking adjacent sorted entries.

Aim

To read names safely, sort them alphabetically, search the sorted list, and identify duplicate entries.

Concept / Theory

With long input, the code drains the remainder of that line and rejects it so the next read begins cleanly. strcmp is case-sensitive and compares character codes lexicographically, so uppercase and lowercase spellings can sort differently. Case-insensitive ordering needs a comparison routine that applies tolower to unsigned-char values character by character.

Procedure / Algorithm

Read with fgets into 30-byte rows, remove the newline, sort with strcmp, binary-search the sorted array, then compare adjacent entries to find duplicates.

Data Analysis / Comparison

Method	Comparison cost	Extra space	Trade-off
Copy names during sort	$O(n^2)$ insertion sort; each compare up to L	O(L) temporary	Simple, but copies strings
Sort indices / pointers	same sort comparisons	O(n) indices / pointers	Less copying; indirect access
qsort pointer array	O(n log n) comparisons × up to L	O(n) pointers	Fast general sort; comparator needed

Program / Source Code

```
#include <stdio.h>
#include <string.h>
#define N 500
#define L 30
void sort_names(char a[][L],int n){
 for(int i=1;i<n;i++){char key[L];strcpy(key,a[i]);int j=i-1;
  while(j>=0&&strcmp(a[j],key)>0){strcpy(a[j+1],a[j]);j--;}
  strcpy(a[j+1],key);}}
int find_name(char a[][L],int n,const char *key,int *cmp){
 int lo=0,hi=n-1;*cmp=0;while(lo<=hi){int m=lo+(hi-lo)/2;(*cmp)++;
  int c=strcmp(a[m],key);if(c==0)return m;if(c<0)lo=m+1;else hi=m-1;}return -1;}
int main(void){char names[N][L];int n=0;
 while(n<N&&fgets(names[n],L,stdin)){size_t z=strlen(names[n]);
  if(z&&names[n][z-1]=='\n')names[n][z-1]='\0';
  else if(z==L-1){int ch=getchar();if(ch!='\n'&&ch!=EOF){while(ch!='\n'&&ch!=EOF)ch=getchar();
   fprintf(stderr,"Name too long (maximum 29 characters)\n");continue;}}
  n++;}
 sort_names(names,n);for(int i=0;i<n;i++){puts(names[i]);
  if(i&&strcmp(names[i],names[i-1])==0)printf("Duplicate: %s\n",names[i]);}
 int cmp,pos=find_name(names,n,"Maya",&cmp);
 printf("Maya index=%d comparisons=%d\n",pos,cmp);return 0;}
```

Output Screenshot

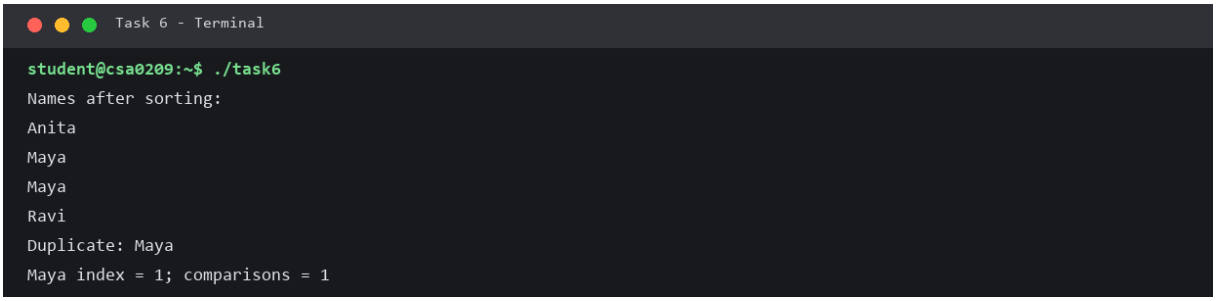


Figure 6.1 - Sample output for task 6.

Analysis / Discussion

For n names, insertion sort may make $O(n^2)$ strcmp calls; binary search makes O(log n) string comparisons, each potentially examining up to L characters. Count each strcmp invocation and each copied character in the experiment for n=100, 500, and 5,000, using generated names of controlled lengths. Sorting indices or pointers reduces record movement: swaps move small indices/pointers rather than up to 30 characters, at the cost of an auxiliary index/pointer array and indirect access.

After sorting, duplicates are adjacent because equal strings occupy neighboring positions in lexicographic order. A name of 29 characters fits with its null terminator; a 30-character name does not fit in a 30-byte row.

Result

After sorting, duplicate names are adjacent. Sorting indices or pointers reduces copying, at the cost of an auxiliary array and indirect access.

TASK 7 - STUDENT COUNCIL ELECTION COUNTING

Task / Problem Statement

Accept candidate count c from 1 to 12 and vote count v from 0 to 5,000. Each vote is valid only when 1 <= vote <= c. Count valid and invalid votes; report candidate totals from highest to lowest; report a winner only if there is no tie at the maximum. Array sizes: count[12] for votes per candidate and result[12] for the candidate order.

Aim

To design an election tally from requirements, reject invalid votes, rank candidates, and detect a tie at the maximum.

Concept / Theory

The count array gives $O(v+c^2)$ total time here: O(v) to classify votes plus $O(c^2)$ insertion sort to rank at most 12 candidates. It uses O(c) auxiliary space. Searching candidate names for every vote costs O(v·c) in a simple list; sorting all votes and counting runs costs O(v log v) time and O(v) storage. A count array indexed by candidate number is direct and compact. For 500 candidates, enlarge the arrays and use an O(c log c) sort if needed; storage remains small. A limitation is that candidate identifiers must be valid bounded integers; write-in candidates require a separate name-matching policy.

To measure 5,000 and 500,000 votes, enlarge the input limit for the second run, generate a reproducible vote file, and time only the counting loop separately from input/output. Record compiler options, hardware, repetitions, and median time. The algorithmic prediction for counting remains O(v); timing values depend on the environment and are intentionally not fabricated here.

Procedure / Algorithm

```
read c, v; require 1<=c<=12 and 0<=v<=5000
initialize count[0..c-1] to zero; valid=invalid=0
repeat v times: read x; if 1<=x<=c count[x-1]++, valid++ else invalid++
create result containing candidate indices 0..c-1
sort result by count descending; break equal-count ties by candidate number ascending
max=count[result[0]]; ties = number of candidates with count == max
print counts, valid, invalid, max; if ties==1 print winner else print tie
```

Program / Source Code

```
#include <stdio.h>
#define CMAX 12
#define VMAX 5000
int main(void){int c,v,count[CMAX]={0},order[CMAX];
 if(scanf("%d%d",&c,&v)!=2||c<1||c>CMAX||v<0||v>VMAX)return 1;
 int valid=0,invalid=0,x;
 for(int i=0;i<v;i++){if(scanf("%d",&x)!=1)return 1;
  if(x>=1&&x<=c){count[x-1]++;valid++;}else invalid++;}
 for(int i=0;i<c;i++)order[i]=i;
 for(int i=1;i<c;i++){int key=order[i],j=i-1;
  while(j>=0&&(count[order[j]]<count[key]||
    (count[order[j]]==count[key]&&order[j]>key))){order[j+1]=order[j];j--;}
  order[j+1]=key;}
 printf("Valid: %d Invalid: %d\n",valid,invalid);
 for(int i=0;i<c;i++)printf("Candidate %d: %d\n",order[i]+1,count[order[i]]);
 int top=count[order[0]],ties=0;for(int i=0;i<c;i++)if(count[i]==top)ties++;
 if(ties==1)printf("Winner: Candidate %d (%d votes)\n",order[0]+1,top);
 else printf("Tie: %d candidates share %d votes\n",ties,top);
 return 0;}
```

Sample Input

```
3 8
1 2 1 0 3 2 4 1
```

Output Screenshot

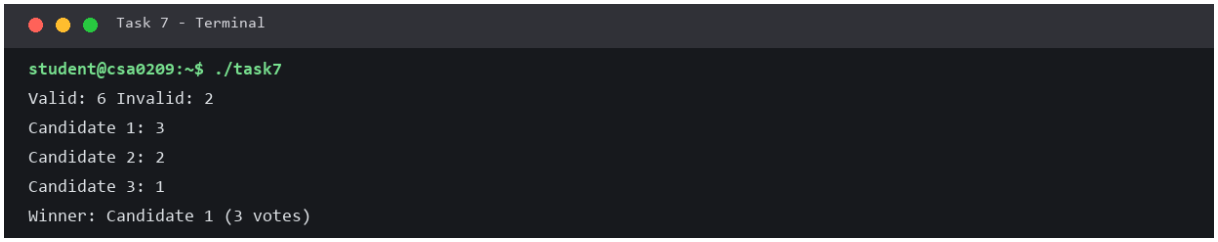


Figure 7.1 - Sample output for task 7.

Analysis / Discussion

Performance measurement

Result

A candidate-indexed count array classifies votes in O(v) time and uses O(c) space; sorting the at most 12 candidate indices ranks the results.

FINAL SUBMISSION CHECKLIST

Checklist item	Status
Student and course details completed	Complete course code; student fields left for entry
Task statement, aim, theory, and algorithm included	Yes
Program code is readable and organized by task	Yes
Output screenshot included	Sample output panels included
Analysis and result written for each task	Yes
Test / measurement considerations included where applicable	Yes
GitHub repository link added if required	Field left for entry